

ALOY: A Local-First AI Operating System for Reliable Multi-Agent Orchestration, Persistent Memory, and Safe Tool Execution

Author: Project Engineering Lead

Target Audience: AI Safety Researchers, Systems Engineers, Fellowship Review Board

Document Status: Technical Design Whitepaper & Codebase Audit (v1.0 Release Candidate)

Abstract

*This paper presents the design and architectural implementation of **ALOY**, an offline-first, local-resource-optimized artificial intelligence operating system (AI OS). Modern Large Language Model (LLM) agents are severely constrained by high cloud dependencies, lack of stateful persistent memory, vulnerability to context-window overflows, and safety risks associated with arbitrary tool execution.*

ALOY resolves these issues by co-designing an asynchronous pipeline-stage conversational engine with a hierarchical memory subsystem, a deterministic agent runtime coordinate-driven by a Finite State Machine (FSM), and a policy-gated tool execution sandbox.

By operating entirely within a local environment via in-process SQLite databases and local LLM backends (Ollama), ALOY guarantees strict data privacy. It mitigates agent reliability risks through transactional workspace locks, file-system snapshots, and automated rollback engines. Furthermore, a background self-evolution loop continuously detects semantic overlap, latency drift, and agent execution failures, proposing self-corrective actions that are applied via a human-in-the-loop review gate.

This paper evaluates the architectural implementation of ALOY and discusses its relevance to practical AI safety engineering.

1. Problem Statement

Autonomous AI assistants built on contemporary LLM frameworks exhibit major engineering limitations that limit their safety and reliability in production:

- Privacy Risks and Cloud Dependence:** Standard agents require sending codebases, system telemetry, and proprietary workspace files to external third-party API providers. This data leakage blocks deployment in enterprise and privacy-sensitive sectors.
- Ephemerality of Cognitive Context:** Chatbots lack native long-term memory. Once a session terminates, the system's cognitive state is lost. Re-injecting historical logs manually causes

significant latency and increases token overhead.

3. **Unreliable and Destructive Tool Execution:** When granted terminal access or script execution capabilities, agents are susceptible to prompt injection, logic errors, and loops. They can permanently destroy local files, delete codebases, or execute malicious commands (e.g., `rm -rf /` or path traversal exploits).
4. **Lack of Continuous Evaluation and Gating:** Agents lack internal feedback loops. They do not automatically evaluate response quality, trace latency degradations, or detect repeated workspace failures, leading to silent degradation.
5. **Context Window Contention:** Running lightweight LLMs locally limits the context window (e.g., 4,096 to 8,192 tokens). Injecting system prompts, tool schemas, long histories, and vector search results leads to context exhaustion, causing model failure or loss of coherence.

ALOY was engineered to address these challenges. It establishes a local runtime that constraints agent execution, preserves persistent semantic memory, and gates operating system modifications through secure pipelines.

2. Design Goals

ALOY is designed around seven primary principles:

- **Local-First Execution:** The system runs completely offline. All model inferences, database operations, vector searches, and code executions occur on local host hardware, avoiding network leakage.
- **Modular Layered Architecture:** The codebase decouples the presentation layer (web interface), the execution engine (FastAPI server), the cognitive pipeline (conversation engine), and the system executor (agent runtime and tool registry).
- **Execution Reliability:** The agent system behaves like a database transaction. Directory changes are tracked via snapshots and restored automatically on task failure.
- **Cognitive Extensibility:** Features an asynchronous stage-based conversation pipeline, making it easy to register new stages (e.g., security scanners, entity extractors) without refactoring the engine.
- **Deterministic Orchestration:** Avoids chaotic agent loops. Multi-agent coordination is governed by a strict state machine with formal state transition rules.
- **Tiered Memory Persistence:** Memory is organized into working, short-term, long-term, and permanent tiers, using decaying importance metrics and semantic clustering to reduce context bloat.
- **Defense-in-Depth Safety:** System operations are blocked by default unless they satisfy strict path sandboxing, verify policy-engine constraints, or obtain explicit human confirmation via an SSE gating loop.

3. System Architecture

ALOY's architecture decouples network coordination, cognitive processing, state tracking, and workspace modifications:

```
graph TD
    Client[Web UI / app.js] <-->|HTTP / SSE| Server[FastAPI Server / server.py]

    subgraph Cognitive Layer
        Server <--> ConvEngine[Conversation Engine]
        ConvEngine --> Pipeline[Asynchronous Pipeline Stages]
        Pipeline --> IntentStage[Intent Classifier]
        Pipeline --> HistoryStage[History Loader]
        Pipeline --> MemoryStage[Memory Retriever]
        Pipeline --> ContextStage[Context Pack Builder]
        Pipeline --> ResponseStage[Response Generator]
    end

    subgraph Memory & Database Layer
        MemoryStage --> MemManager[Memory Manager]
        MemManager --> Cache[LRU Retrieval Cache]
        MemManager --> VecStore[Vector Store / vec0]
        MemManager --> TextStore[FTS5 Search / memories_fts]
        MemManager --> DB[(SQLite DB / aloy.db)]
    end

    subgraph Execution & Security Layer
        Server <--> AgentRuntime[Agent Runtime]
        AgentRuntime --> FSM[Session FSM]
        AgentRuntime --> TaskQueue[Agent Task Queue]
        AgentRuntime --> ToolSystem[Tool System]
        ToolSystem --> Sandbox[Sandbox Validator]
        ToolSystem --> PolicyEngine[Policy Engine]
        ToolSystem --> Confirmation[Confirmation Workflow]
        ToolSystem --> Tools[Tool Registry / impl/]
        Tools <--> Workspace[(User Workspace Filesystem)]
        AgentRuntime --> Rollback[Advanced Rollback Engine]
        Rollback --> Snapshot[Workspace Snapshot Manager]
    end

    subgraph Self-Evolution Layer
        EvoService[Evolution Engine] --> IssueDetector[Issue Detector]
        IssueDetector --> DB
        EvoService --> ReviewGate[Review Gate]
        ReviewGate --> AgentRuntime
    end
```

The lifespans and instances of these subsystems are coordinated via the FastAPI app state in `[server.py](file:///c:/Users/DHANUSH%20ANBU/Desktop/MYAI%20FINAL/api/server.py)`.

4. Conversation Pipeline

The conversation logic in `[engine.py](file:///c:/Users/DHANUSH%20ANBU/Desktop/MYAI%20FINAL/conversation/engine.py)` implements the Pipeline Stage pattern. It processes a user message through a series of discrete, asynchronous stages to construct a context-rich prompt and generate a streamed response.

4.1 Pipeline Stages

The pipeline registers the following stages in sequence:

1. `**IntentDetectionStage**`
`([intent.py](file:///c:/Users/DHANUSH%20ANBU/Desktop/MYAI%20FINAL/conversation/intent.py))`: Performs regex pattern matching to classify the query's intent (e.g., ``memory_query``, ``coding_request``, ``tool_request``). If patterns do not match, it routes the prompt to the fallback model router for classification.
2. `**HistoryLoadStage**`
`([context_builder.py](file:///c:/Users/DHANUSH%20ANBU/Desktop/MYAI%20FINAL/conversation/context_builder.py))`: Retrieves the last `$N$` turns (default 10) of conversation history from the SQLite database to maintain conversational context.
3. `**MemoryRetrievalStage**`
`([context_builder.py](file:///c:/Users/DHANUSH%20ANBU/Desktop/MYAI%20FINAL/conversation/context_builder.py))`: Queries the memory database using keyword terms extracted from the query, pulling candidate memory records.
4. `**ContextBuildStage**`
`([context_builder.py](file:///c:/Users/DHANUSH%20ANBU/Desktop/MYAI%20FINAL/conversation/context_builder.py))`: Wires context data, triggers web searches if live data is needed, builds system instructions, and formats everything into a structured prompt.
5. `**ResponseGenerationStage**`
`([response.py](file:///c:/Users/DHANUSH%20ANBU/Desktop/MYAI%20FINAL/conversation/response.py))`: Routes the final prompt payload to the designated local model, yielding an asynchronous token generator.

4.2 Token Budgeting and Context Pruning

To prevent context overflow, the `TokenBudgetManager`

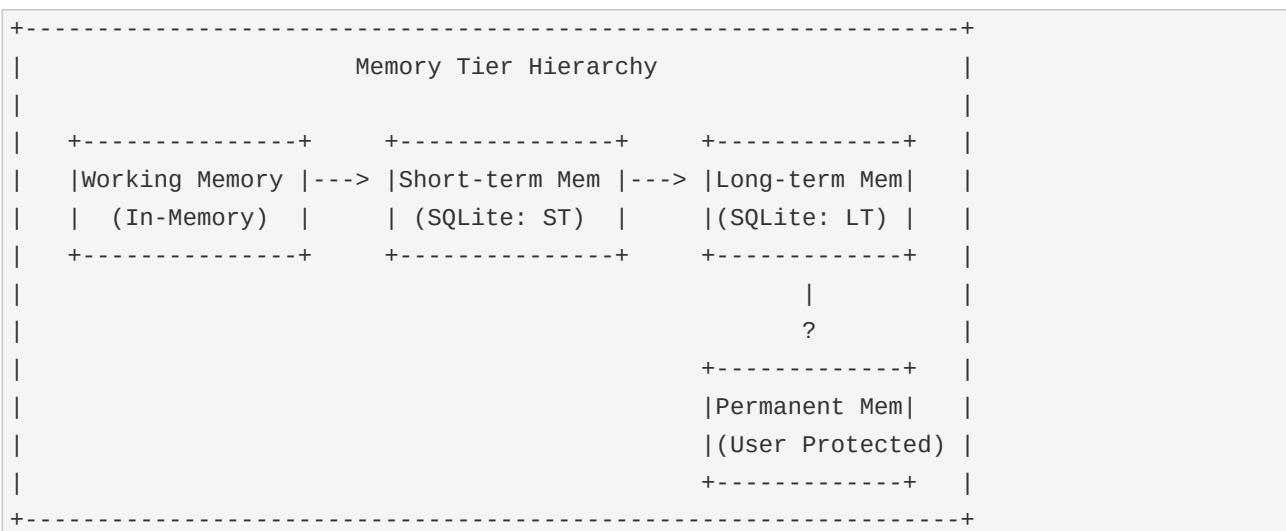
([token_manager.py](file:///c:/Users/DHANUSH%20ANBU/Desktop/MYAI%20FINAL/conversation/token_manager.py)) manages token allocations:

$$\text{Total Context Budget (e.g., 8192)} = \text{System Prompt} + \text{Identity Context} + \text{Retrieved Memories} + \text{Tool Schemas} + \text{Chat History}$$

If the calculated token count of the formatted history exceeds its allocation, a loop prunes older history turns or summarizes them, keeping the prompt safely within the model's context limits.

5. Memory Architecture

ALOY implements a tiered memory system that mimics biological memory consolidation:



- **Working Memory:** Local in-memory context representing the active session.
- **Short-Term Memory:** Persistent records in SQLite initialized with high decay rates.
- **Long-Term Memory:** Promoted short-term records that have been accessed multiple times.
- **Permanent Memory:** Critical facts protected from decay (e.g., project details, user identity).

5.1 Hybrid Retrieval (FTS5 + vec0)

Retrieved memories are queried using two complementary search mechanisms:

1. **Full-Text Search (FTS5):** Querying ``memories_fts`` using prefix matching and Porter stemming to locate keyword matches quickly.
2. **Vector Similarity (``sqlite-vec``):** Computing Cosine Similarity between a query's embedding and stored memory embeddings:

$$\text{Cosine Similarity} = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

The `RetrievalEngine` ([retrieval.py](file:///c:/Users/DHANUSH%20ANBU/Desktop/MYAI%20FINAL/memory/retrieval.py)) merges both sets, deduplicates by memory ID, and ranks the results to return the most relevant snippets.

5.2 Consolidation and Decay Engine

The `ConsolidationEngine` ([consolidation.py](file:///c:/Users/DHANUSH%20ANBU/Desktop/MYAI%20FINAL/memory/consolidation.py)) runs periodically during idle times:

- 1. **Memory Promotion:** Promotes short-term memories with an `access_count >= promotion_threshold` (default 5) to long-term memory.
- 2. **Decay Engine:** Reduces the importance scores of unused memories over time using an exponential decay formula:

$$I_{\text{new}} = I_{\text{old}} \times e^{-\lambda t}$$

Where λ represents the decay rate and t is the idle time in hours. If a memory's importance score falls below a threshold (0.05), the system compresses and archives it.

- 3. **Semantic Consolidation:** Computes a multi-factor merge score to consolidate duplicate memories:
$$\text{Merge Score} = 0.35 \times \text{Cosine Sim} + 0.20 \times \text{Imp Delta} + 0.15 \times \text{Avg Conf} + 0.15 \times \text{Avg Recency} + 0.10 \times \text{Tag Overlap} + 0.05 \times \text{Graph Link}$$

Pairs matching a similarity threshold ≥ 0.92 are merged into a single consolidated memory.

6. Multi-Model Orchestration

ALOY routes tasks to specialized models using the `ModelRouter` ([router.py](file:///c:/Users/DHANUSH%20ANBU/Desktop/MYAI%20FINAL/models/router.py)) to balance performance, latency, and system memory usage:

Task Type	Primary Model	Fallback Model	Execution Priority
simple_chat	phi4-mini:latest	qwen2.5-coder:14b	High (User Facing)
coding_request	qwen2.5-coder:14b	phi4-mini:latest	High (User Facing)
reasoning_request	deepseek-r1:14b	qwen2.5-coder:14b	Medium (User/Agent)
memory_generation	phi4-mini:latest	None	Low (Background)

6.1 Task Specialization and Fallback Rules

- **phi4-mini:** Handles lightweight tasks like memory generation and simple chat, minimizing system memory usage.
- **qwen2.5-coder:** Handles code generation, file editing, and test execution tasks.
- **deepseek-r1:** Handles deep logical reasoning, debugging, and complex math.
- **Fallback Logic:** If a primary model fails or times out, the router automatically catches the exception and falls back to a secondary model to prevent pipeline crashes.

7. Agent Framework

ALOY implements a deterministic agent runtime governed by a strict Finite State Machine (FSM) to coordinate multi-agent workflows safely:

```
stateDiagram-v2
    [*] --> PLANNING : start_session()
    PLANNING --> EXECUTING : get_next_task()
    EXECUTING --> EXECUTING : run_step()
    EXECUTING --> PAUSED : pause_session()
    PAUSED --> EXECUTING : resume_session()
    EXECUTING --> ROLLEDBACK : error / rollback()
    ROLLEDBACK --> EXECUTING : retry()
    EXECUTING --> COMPLETED : all_tasks_done
    EXECUTING --> FAILED : max_retries_exceeded
```

7.1 State-Transition Reliability Mechanics

- **Workspace Locks:** Prior to starting execution, the `WorkspaceLockManager` ([lock.py](file:///c:/Users/DHANUSH%20ANBU/Desktop/MYAI%20FINAL/agent/lock.py)) acquires a lock on the directory. If another session holds the lock, execution is blocked to prevent concurrent write conflicts.
- **Workspace Snapshots:** Before a worker agent runs a task step, the `WorkspaceSnapshotManager` ([snapshot.py](file:///c:/Users/DHANUSH%20ANBU/Desktop/MYAI%20FINAL/agent/snapshot.py)) creates a zip backup of the directory.
- **Rollback Engine:** If a task fails or violates a security policy, the `AdvancedRollbackEngine` ([rollback.py](file:///c:/Users/DHANUSH%20ANBU/Desktop/MYAI%20FINAL/security/rollback.py)) extracts the latest snapshot, restoring the workspace to a clean state.

8. Tool Execution Framework

The `ToolSystem` ([system.py](file:///c:/Users/DHANUSH%20ANBU/Desktop/MYAI%20FINAL/tools/system.py)) manages tool registry discovery, validation, and execution:

- **File Editor Tool:** Edits code files using diff blocks, avoiding the token overhead of rewriting entire files.
- **Terminal Tool:** Runs command line scripts under strict timeout limits to prevent processes from hanging.
- **Docker Tool:** Executes code in isolated Docker containers, sandboxing the execution environment.

8.1 Safety Gating & Sandbox Policies

- **Directory Sandboxing:** The ``Sandbox`` class ([sandbox.py](file:///c:/Users/DHANUSH%20ANBU/Desktop/MYAI%20FINAL/security/sandbox.py)) checks all file paths. If an agent tries to access directories outside the workspace root, the operation is blocked.
- **Action Policy Gating:** The ``PolicyEngine`` ([policy.py](file:///c:/Users/DHANUSH%20ANBU/Desktop/MYAI%20FINAL/security/policy.py)) evaluates requested actions. Read actions are allowed, while write and execute actions require user approval.
- **Confirmation Workflow:** When an action requires approval, the system pauses execution and sends an SSE alert to the UI. Execution remains paused until the user clicks "Approve", resuming the task safely.

9. Backend Engineering

The backend is built with FastAPI, utilizing async features for high concurrency:

- **Async Connection Pools:** Connects to SQLite using an async lifecycle coordinator. Writes are serialized using python locks, while read-only connections are pooled.
- **SSE Event Streaming:** Pushes model response tokens, tool outputs, and confirmation requests to the UI in real-time.
- **Lifespan Management:** Creates ``running.tmp`` on startup. If the server exits cleanly, the file is removed. If it crashes, the file remains, allowing the system to detect and recover from crashes on the next boot.

10. Database Design

ALOY stores all data locally in a structured SQLite database:

```
-- Core memories table
CREATE TABLE memories (
  id TEXT PRIMARY KEY,
  type TEXT NOT NULL,
  category TEXT,
  tier TEXT NOT NULL DEFAULT 'short_term',
  content TEXT NOT NULL,
  summary TEXT,
  source TEXT,
  confidence REAL NOT NULL DEFAULT 0.5,
  importance REAL NOT NULL DEFAULT 0.5,
  access_count INTEGER NOT NULL DEFAULT 0,
  last_accessed_at TEXT,
  decay_rate REAL NOT NULL DEFAULT 0.01,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL,
  expires_at TEXT,
  archived_at TEXT,
  is_protected INTEGER NOT NULL DEFAULT 0,
  metadata TEXT,
  embedding BLOB
);

-- Virtual table for vector search
CREATE VIRTUAL TABLE memory_embeddings USING vec0(
  embedding float[768]
);

-- Virtual table for full-text search
CREATE VIRTUAL TABLE memories_fts USING fts5(
  content, summary, category,
  content='memories',
  content_rowid='rowid',
  tokenize='porter unicode61'
);
```

10.1 Indexes and WAL

- **Partial Indexing:** Created an index on `archived\_at` where `archived\_at IS NULL` to speed up active memory queries.
- **WAL Mode:** Enabled Write-Ahead Logging to support concurrent reads while writes are active, preventing database locks.

11. Validation Framework

The validation framework ([mega_audit.py](file:///c:/Users/DHANUSH%20ANBU/Desktop/MYAI%20FINAL/validation/mega_audit.py)) acts as a local continuous integration (CI) suite for testing AI quality:

- **Automated Evaluation:** Evaluates response quality across 18 criteria (e.g., identity integrity, memory honesty, safety filters, formatting).
- **Hallucination Detection:** Flags instances where the model fabricates information or fails to admit lack of data.
- **Regression Auditing:** Executes test suites and calculates pass rates, latency metrics, and release readiness scores to prevent system regressions.

12. Reliability Engineering

ALOY implements several mechanisms to guarantee system reliability:

- **Transactional Rollbacks:** Restores workspace files to a clean snapshot if an agent task fails.
- **Workspace Isolation:** Uses directory locks to prevent concurrent agent sessions from modifying the same files.
- **Process Gating:** Sets execution timeouts on terminal commands to prevent hanging processes from exhausting host resources.
- **State Reconciliation:** Resets interrupted tasks to failed or pending status if the server crashes, clearing stale workspace locks.

13. Security Considerations

ALOY's security architecture centers on control and containment:

- **Local Data Boundaries:** By running models and storage locally, private data and keys are never exposed to external networks.
- **Strict Sandboxing:** Blocks path traversal attempts, preventing agents from accessing files outside the workspace.
- **Human Oversight Gating:** Pauses risky executions and prompts the user for approval, ensuring developers retain control over file writes and command runs.

14. Performance Optimizations

- **Async Embedding Generation:** Offloads expensive vector embedding calculations to background threads.
- **LRU Retrieval Cache:** Caches up to 100 memory queries, lowering retrieval latency.
- **Ollama Keep-Alive:** Keeps models resident in GPU memory to prevent loading latencies during user queries.
- **SQLite PRAGMA Tuning:** Allocates up to 64MB of RAM for database caching and optimizes write operations.

15. Engineering Challenges

15.1 Context Window Overflow

- **The Problem:** Local models have small context windows. Background logs, memory search results, and chat history can easily overflow these limits.
- **The Solution:** Implemented the `TokenBudgetManager`. It monitors token counts, prunes old history entries, and summarizes text blocks to fit prompt payloads within context limits.

15.2 Concurrent Workspace Corruption

- **The Problem:** Multiple agents making concurrent file modifications can easily corrupt the directory.
- **The Solution:** Implemented the `WorkspaceLockManager` and `WorkspaceSnapshotManager`. Directories are locked during active sessions, and changes are reverted to the pre-task snapshot if an error occurs.

16. AI Safety Relevance

ALOY serves as a practical implementation of AI safety engineering principles:

- **Controlled Autonomy:** Limits agent actions using a strict Finite State Machine. Agents cannot execute arbitrary loops or state transitions.
- **Path Containment:** Restricts file system tools to the active workspace, protecting core OS files.
- **Active Gating:** Prevents unauthorized write and execution operations, requiring human confirmation by default.
- **Reliable Evaluation:** Features an automated evaluation framework to test safety, routing, and

quality metrics before system release.

17. Future Work

- **Formal Verification:** Implement formal state-transition checks for the agent runtime FSM.
 - **Uncertainty Estimation:** Integrate confidence-scoring metrics to detect when models generate low-confidence responses.
 - **Memory Quality Profiling:** Track memory fragmentation and optimize consolidation algorithms.
 - **Automated Rollback Logs:** Improve rollback logging to help developers debug failed agent sessions.
-

18. Personal Contributions

Architecture & Infrastructure

- Designed the async pipeline-stage conversation flow.
- Wrote the Event Bus messaging system.
- Implemented the FastAPI lifespan manager and crash recovery logic.

AI & Memory

- Built the local model router and fallback selection rules.
- Wrote the tiered memory consolidation engine.
- Integrated hybrid memory retrieval using FTS5 and `sqlite-vec`.

Security & Agent Core

- Coded the FSM agent runtime and task queue system.
- Built the directory lock manager and rollback engine.
- Programmed the path validation sandbox and policy confirmation workflow.

Verification & Frontend

- Built the validation regression test suite.
 - Developed the HTML/JS frontend chat client and streaming renderer.
-

19. Lessons Learned

- **Local Resource Optimization:** Local models require strict context budgeting. Managing context limits is key to maintaining response coherence.
 - **FSM-Governed Agent Reliability:** Modeling agent tasks as formal state transitions simplifies error recovery and rollback logic.
 - **User Confirmation UI:** Keeping the confirmation loop responsive prevents developer friction while maintaining system safety.
-

20. Conclusion

ALOY demonstrates the feasibility of running a secure, reliable, and persistent AI operating system locally on consumer hardware. By integrating FSM agent coordination, tiered semantic memory, path sandboxing, and human-in-the-loop gating, it addresses critical security and reliability challenges. This architecture offers a practical template for designing trustworthy local AI agents.

ALOY Technical Fellowship Work Sample Submission | v1.0 Release Candidate